

Homework 03

STAT 432 — Fall 2026 · Owen Park

```
In [1]: suppressPackageStartupMessages({
  library(ggplot2)
  library(dplyr)
})
set.seed(432)
options(repr.plot.width = 7.5, repr.plot.height = 3.4)
```

Question 1: Build an explain-to-me skill

No code here. Skill text goes in the last cell of this notebook.

Question 2: Ridge regression with nearly collinear predictors

$n = 100$, seed 43231. $L, U1, U2, X3, X4 \sim N(0,1)$. $X1 = L + 0.04 \cdot U1$, $X2 = L + 0.04 \cdot U2$. Center + scale each column to mean 0, average square 1 (divide by n). Keep X fixed. $\beta = (1.5, 1.5, 1, 0)$. $\mu = X\beta$. $y = \mu + \varepsilon$, $y = \mu + \varepsilon$. $\lambda \in \{0, 0.02, 0.2\}$. Ridge: $\hat{\beta} = (X^T X + n\lambda I)^{-1} X^T (y - \bar{y})$, intercept = $\bar{y}$.

(a) Generate X once. Report $\text{cor}(X1, X2)$, largest and smallest singular value. Explain why $\beta_1 - \beta_2$ is much more variable than $\beta_1 + \beta_2$.

```
In [2]: set.seed(43231)
n <- 100
L <- rnorm(n)
U1 <- rnorm(n)
U2 <- rnorm(n)
X1 <- L + 0.04 * U1
X2 <- L + 0.04 * U2
X3 <- rnorm(n)
X4 <- rnorm(n)
X <- cbind(X1, X2, X3, X4)

X <- sweep(X, 2, colMeans(X))
X <- sweep(X, 2, sqrt(colMeans(X^2)), "/")

beta <- c(1.5, 1.5, 1, 0)
mu <- drop(X %*% beta)

ridge_fit <- function(X, ytil, lam) {
  p <- ncol(X)
  A <- crossprod(X) + n * lam * diag(p) # X'X + n*lambda*I
  drop(solve(A, crossprod(X, ytil))) # A^{-1} X'ytil, as a plain vector
}

cor(X)[1, 2]
d <- svd(X)$d
max(d); min(d)
```

0.997705689967843

14.1414804265881

0.475357498832516

The correlation between the first two standardized columns is about 0.998. The largest singular value of $\mathbf{X}$ is about 14.1 and the smallest is about 0.48, so the largest is roughly 30 times the smallest.

For OLS, $\text{Var}(\hat{\beta}) = \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1} = \sigma^2 \mathbf{V} \mathbf{D}^{-2} \mathbf{V}^T$, so the variance of a linear combination $\mathbf{c}^T \hat{\beta}$ is $\sigma^2 \sum_j (\mathbf{v}_j^T \mathbf{c})^2 / d_j^2$. The right singular vector for the smallest singular value is almost

exactly $(1, -1, 0, 0)/\sqrt{2}$, and the one for the largest is almost exactly $(1, 1, 0, 0)/\sqrt{2}$. So the contrast $\hat{\beta}_1 - \hat{\beta}_2$ lives in the direction with the smallest d_j and has variance about $2\sigma^2/d_{\min}^2 \approx 8.9$, while $\hat{\beta}_1 + \hat{\beta}_2$ lives in the direction with the largest d_j and has variance about $2\sigma^2/d_{\max}^2 \approx 0.01$. The ratio is $(d_{\max}/d_{\min})^2$, close to 900.

Intuitively, X_1 and X_2 are almost the same column, so the data can pin down their sum but say almost nothing about how to split that sum between the two. Any split gives nearly the same fitted values, so the difference is nearly free to move.

(b) Repeat 200x: draw ϵ, ϵ^* , fit all three λ on the same pair. For each λ report mean and sd of $\beta_1, \beta_2, \beta_1 + \beta_2, \beta_1 - \beta_2$, plus average train MSE and test MSE.

```
In [3]: lambdas <- c(0, 0.02, 0.2)
R <- 200

coefs <- array(NA, dim = c(R, 4, length(lambdas))) # rep x coefficient x lambda
mse <- array(NA, dim = c(R, 2, length(lambdas))) # rep x (train, test) x lambda

for (r in 1:R) {
  y <- mu + rnorm(n)
  ystar <- mu + rnorm(n)
  b0 <- mean(y)
  ytil <- y - b0
  for (k in seq_along(lambdas)) {
    bhat <- ridge_fit(X, ytil, lambdas[k])
    yhat <- b0 + drop(X %*% bhat)
    coefs[r, , k] <- bhat
    mse[r, , k] <- c(mean((y - yhat)^2), mean((ystar - yhat)^2))
  }
}

# summarize: for each lambda, mean and sd of b1, b2, b1 + b2, b1 - b2, plus average MSEs
rows <- lapply(seq_along(lambdas), function(k) {
  B <- coefs[, , k]
  Q <- cbind(b1 = B[, 1], b2 = B[, 2], sum = B[, 1] + B[, 2], diff = B[, 1] - B[, 2])
  data.frame(
    lambda = lambdas[k],
    b1_mean = mean(Q[, "b1"]), b1_sd = sd(Q[, "b1"]),
    b2_mean = mean(Q[, "b2"]), b2_sd = sd(Q[, "b2"]),
    sum_mean = mean(Q[, "sum"]), sum_sd = sd(Q[, "sum"]),
    diff_mean = mean(Q[, "diff"]), diff_sd = sd(Q[, "diff"]),
    mse_train = mean(mse[, 1, k]),
    mse_test = mean(mse[, 2, k])
  )
})
summary_b <- do.call(rbind, rows)
knitr::kable(summary_b, digits = 3)
```

	lambda	b1_mean	b1_sd	b2_mean	b2_sd	sum_mean	sum_sd	diff_mean	diff_sd
0	0.00	1.496	1.544	1.510	1.549	3.005	0.095	-0.014	3.09
1	0.02	1.487	0.162	1.489	0.166	2.976	0.094	-0.002	0.31
4	0.20	1.368	0.046	1.369	0.047	2.737	0.086	-0.001	0.03
4	1.077	1.133							

(c) Interpret with bias and variance. Why can ridge stabilize the coefficients a lot while the fitted values barely change?

Bias. At $\lambda = 0$ the estimates are unbiased: the means of $\hat{\beta}_1, \hat{\beta}_2$ and their sum are essentially 1.5, 1.5 and 3. At $\lambda = 0.02$ the means barely move (about 1.49 and 2.98). At $\lambda = 0.2$ the shrinkage is visible: the means drop to about 1.37 and the sum to about 2.74, so ridge is now pulling the slopes toward zero and away from the truth.

Variance. The variance story is all in the contrast. At $\lambda = 0$ the sd of $\hat{\beta}_1 - \hat{\beta}_2$ is about 3.1, which matches $\sqrt{2\sigma^2/d_{\min}^2} \approx \sqrt{8.9}$ from part (a), while the sd of the sum is only about 0.1. A small penalty, $\lambda = 0.02$, cuts the sd of the difference by a factor of ten (to about 0.31) and each individual slope's sd from about 1.5 to about 0.16, with almost no bias. At $\lambda = 0.2$ the difference is essentially pinned at zero (sd about 0.03), but the bias above now costs more than the variance it saves.

Test MSE. Test MSE is about 1.06 at $\lambda = 0$, 1.05 at $\lambda = 0.02$, and 1.13 at $\lambda = 0.2$. So $\lambda = 0.02$ wins slightly (variance reduction with negligible bias), and $\lambda = 0.2$ loses (bias dominates). But the striking fact is how flat these numbers are compared with the coefficient variances: the sd of an individual OLS slope is about 1.5, yet OLS test MSE is only about 0.01 worse than the best ridge fit.

Why the fitted values barely change. The fitted values are $\hat{\beta}_0 \mathbf{1} + \mathbf{X}\hat{\beta}$, and $\mathbf{X}$ maps the direction $\mathbf{v}_{\min} \approx (1, -1, 0, 0)/\sqrt{2}$ to a vector of length $d_{\min} \approx 0.48$, tiny compared with $d_{\max} \approx 14$. Because $X_1 \approx X_2$, moving weight from $\hat{\beta}_1$ to $\hat{\beta}_2$ hardly changes $X_1\hat{\beta}_1 + X_2\hat{\beta}_2$. So the huge OLS variance along $\mathbf{v}_{\min}$ is almost invisible in $\mathbf{X}\hat{\beta}$, and killing it with ridge is almost invisible there too. In SVD terms ridge multiplies the fitted component in direction j by $d_j^2/(d_j^2 + n\lambda)$: at $\lambda = 0.02$ that factor is about 0.10 for the $d_{\min}$ direction but about 0.99 for the $d_{\max}$ direction. Ridge therefore rewrites the coefficients (which live in the $\mathbf{v}_{\min}$ direction) while leaving the part of the fit that carries signal (the $\mathbf{v}_{\max}$ direction, i.e. the sum) essentially untouched.

Question 3: What does ridge regression shrink?

(a) Derive $(\mathbf{X}^T\mathbf{X} + n\lambda\mathbf{I})\hat{\beta} = \mathbf{X}^T\tilde{\mathbf{y}}$ from $L(\beta) = (1/2n)\|\tilde{\mathbf{y}} - \mathbf{X}\beta\|^2 + (\lambda/2)\|\beta\|^2$. Why unique for $\lambda > 0$ even if $\mathbf{X}$ is rank deficient or $p > n$?

Expand the squared norm. With $\tilde{\mathbf{y}}$ and $\mathbf{X}$ fixed,

$$L_\lambda(\beta) = \frac{1}{2n} \left(\tilde{\mathbf{y}}^T \tilde{\mathbf{y}} - 2\beta^T \mathbf{X}^T \tilde{\mathbf{y}} + \beta^T \mathbf{X}^T \mathbf{X} \beta \right) + \frac{\lambda}{2} \beta^T \beta.$$

Differentiate with respect to β , using $\nabla_\beta(\beta^T \mathbf{a}) = \mathbf{a}$ and $\nabla_\beta(\beta^T \mathbf{A} \beta) = 2\mathbf{A}\beta$ for symmetric $\mathbf{A}$:

$$\nabla L_\lambda(\beta) = \frac{1}{n} \left(\mathbf{X}^T \mathbf{X} \beta - \mathbf{X}^T \tilde{\mathbf{y}} \right) + \lambda \beta.$$

Set the gradient to zero and multiply by n :

$$\mathbf{X}^T \mathbf{X} \beta + n\lambda \beta = \mathbf{X}^T \tilde{\mathbf{y}} \iff \left(\mathbf{X}^T \mathbf{X} + n\lambda \mathbf{I}_p \right) \hat{\beta}_\lambda = \mathbf{X}^T \tilde{\mathbf{y}}.$$

Uniqueness for $\lambda > 0$. $\mathbf{X}^T \mathbf{X}$ is symmetric positive semidefinite, so its eigenvalues are ≥ 0 . Adding $n\lambda \mathbf{I}_p$ shifts every eigenvalue up by $n\lambda$, so the eigenvalues of $\mathbf{X}^T \mathbf{X} + n\lambda \mathbf{I}_p$ are all $\geq n\lambda > 0$. The matrix is therefore positive definite and invertible, and the normal equation has exactly one solution,

$$\hat{\beta}_\lambda = \left(\mathbf{X}^T \mathbf{X} + n\lambda \mathbf{I}_p \right)^{-1} \mathbf{X}^T \tilde{\mathbf{y}}.$$

This does not use the rank of $\mathbf{X}$ at all. If $\mathbf{X}$ is rank deficient or $p > n$, then $\mathbf{X}^T \mathbf{X}$ has zero eigenvalues and the OLS normal equation ($\lambda = 0$) has infinitely many solutions, but the ridge matrix still has smallest eigenvalue $n\lambda > 0$. Equivalently, the Hessian

$\nabla^2 L_\lambda = \frac{1}{n} \mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_p$ is positive definite, so L_λ is strictly convex and its stationary point is the unique global minimizer.

(b) With $\mathbf{X} = \mathbf{U} \mathbf{D} \mathbf{V}^\top$, show ridge scales the j -th fitted component by $\rho_j = d_j^2 / (d_j^2 + n\lambda)$. Then $n = 100$, $\lambda = 0.01$, $\mathbf{d} = (10, 2, 0.2)$: compute the three ρ_j and $\text{df}_{\text{eff}} = 1 + \sum \rho_j$.

Write $\mathbf{X} = \mathbf{U}_r \mathbf{D}_r \mathbf{V}_r^\top$ with $\mathbf{U}_r^\top \mathbf{U}_r = \mathbf{V}_r^\top \mathbf{V}_r = \mathbf{I}_r$. Then $\mathbf{X}^\top \mathbf{X} = \mathbf{V}_r \mathbf{D}_r^2 \mathbf{V}_r^\top$ and $\mathbf{X}^\top \tilde{\mathbf{y}} = \mathbf{V}_r \mathbf{D}_r \mathbf{U}_r^\top \tilde{\mathbf{y}}$. Because $\mathbf{V}_r$ has orthonormal columns, $(\mathbf{V}_r \mathbf{D}_r^2 \mathbf{V}_r^\top + n\lambda \mathbf{I}_p) \mathbf{V}_r = \mathbf{V}_r (\mathbf{D}_r^2 + n\lambda \mathbf{I}_r)$, so the ridge solution from part (a) is

$$\hat{\boldsymbol{\beta}}_\lambda = \mathbf{V}_r (\mathbf{D}_r^2 + n\lambda \mathbf{I}_r)^{-1} \mathbf{D}_r \mathbf{U}_r^\top \tilde{\mathbf{y}}.$$

(Any component of $\hat{\boldsymbol{\beta}}_\lambda$ orthogonal to the columns of $\mathbf{V}_r$ would be pure penalty with no fit benefit, so it is zero.) The fitted centered response is therefore

$$\mathbf{X} \hat{\boldsymbol{\beta}}_\lambda = \mathbf{U}_r \mathbf{D}_r \mathbf{V}_r^\top \mathbf{V}_r (\mathbf{D}_r^2 + n\lambda \mathbf{I}_r)^{-1} \mathbf{D}_r \mathbf{U}_r^\top \tilde{\mathbf{y}} = \sum_{j=1}^r \frac{d_j^2}{d_j^2 + n\lambda} \mathbf{u}_j (\mathbf{u}_j^\top \tilde{\mathbf{y}}).$$

OLS ($\lambda = 0$) keeps each component $\mathbf{u}_j (\mathbf{u}_j^\top \tilde{\mathbf{y}})$ in full; ridge multiplies the j th one by $\rho_j(\lambda) = d_j^2 / (d_j^2 + n\lambda) \in (0, 1)$, and directions with smaller d_j are shrunk more. With $n = 100$, $\lambda = 0.01$, so $n\lambda = 1$:

$$\rho_1 = \frac{100}{101} \approx 0.990, \quad \rho_2 = \frac{4}{5} = 0.800, \quad \rho_3 = \frac{0.04}{1.04} \approx 0.038, \\ \text{df}_{\text{eff}} = 1 + \rho_1 + \rho_2 + \rho_3 \approx 2.83.$$

In [4]:

```
n <- 100
lam <- 0.01
d <- c(10, 2, 0.2)
rho <- d^2 / (d^2 + n * lam)
rho
1 + sum(rho) # effective df, including the unpenalized intercept
```

```
0.990099009900999 * 0.8 * 0.0384615384615385
```

```
2.82856054836253
```

(c) Which direction shrinks most? Why does that cut prediction variance? When would it cause large bias?

The direction with the smallest singular value, $d_3 = 0.2$, is shrunk the most: its factor is $\rho_3 \approx 0.038$, so ridge keeps only about 4% of the fitted component along $\mathbf{u}_3$, while it keeps 99% along $\mathbf{u}_1$ and 80% along $\mathbf{u}_2$. A small d_j means $\mathbf{X}$ has very little spread in the direction $\mathbf{v}_j$ (in Question 2 this was the $\hat{\beta}_1 - \hat{\beta}_2$ contrast); the data say almost nothing about that direction, so the OLS coefficient along $\mathbf{v}_j$ is $\mathbf{u}_j^\top \tilde{\mathbf{y}} / d_j$, i.e. noise divided by a tiny number.

Why this cuts prediction variance. Write $\tilde{\mathbf{y}} = \boldsymbol{\mu} + \boldsymbol{\epsilon}$. The fitted component along $\mathbf{u}_j$ is $\rho_j \mathbf{u}_j \mathbf{u}_j^\top \tilde{\mathbf{y}}$, whose variance is $\sigma^2 \rho_j^2$ (per direction). OLS pays σ^2 for every one of the r directions, whether or not they carry signal. Ridge pays $\sigma^2 \rho_j^2$, which for $\mathbf{u}_3$ is about $0.0015\sigma^2$ instead of σ^2 . Total variance of the fit drops from $r\sigma^2$ (plus the intercept) to $\sigma^2 \sum_j \rho_j^2$, and the biggest saving comes exactly from the most-shrunk direction.

When it causes large bias. The price is bias $(1 - \rho_j) \mathbf{u}_j^T \boldsymbol{\mu}$ in direction $\mathbf{u}_j$. For $\mathbf{u}_3$ that is 96% of whatever signal lives there. Shrinking is nearly free when $\mathbf{u}_3^T \boldsymbol{\mu}$ is small, i.e. when the true mean has little energy along the low-variance direction, which is the usual case. It produces substantial prediction bias when the true signal is concentrated along $\mathbf{u}_3$: equivalently, when $\boldsymbol{\beta}$ has a large component along $\mathbf{v}_3$ so that $d_3(\mathbf{v}_3^T \boldsymbol{\beta})$ is not negligible. Then ridge throws away most of a real effect, and the squared bias $(0.96)^2 (\mathbf{u}_3^T \boldsymbol{\mu})^2$ can exceed the variance it saved.

Question 4: Ridge regression and optimization

■ $p = 2$. Eigenvalues of $\mathbf{A}_0 = \mathbf{X}^T \mathbf{X} / n$ are 9 and 0.01. $\lambda = 0.09$. $\mathbf{A}_\lambda = \mathbf{A}_0 + \lambda \mathbf{I}$.

(a) Condition numbers of $\mathbf{A}_0$ and $\mathbf{A}_\lambda$. What does the change mean for the geometry of the objective?

The question supplies the eigenvalues directly, so nothing is estimated from data here. The condition number is the ratio of the largest to the smallest eigenvalue:

$$\kappa(\mathbf{A}_0) = \frac{9}{0.01} = 900, \quad \kappa(\mathbf{A}_\lambda) = \frac{9 + 0.09}{0.01 + 0.09} = \frac{9.09}{0.10} = 90.9.$$

Adding $\lambda \mathbf{I}_p$ shifts both eigenvalues by λ but leaves the eigenvectors alone. The shift is negligible for the large eigenvalue (9 to 9.09) and enormous for the small one (0.01 to 0.10), so the ratio drops by a factor of about ten.

Geometry. L_λ is a quadratic whose level sets are ellipses centered at $\hat{\boldsymbol{\beta}}_\lambda$ with axes along the eigenvectors of $\mathbf{A}_\lambda$ and axis lengths proportional to $1/\sqrt{\text{eigenvalue}}$. The ratio of the long axis to the short axis is $\sqrt{\kappa}$: about 30 for OLS and about 9.5 for ridge. The OLS objective is a long narrow valley (almost flat along the small-eigenvalue direction, steep across it); ridge makes the bowl noticeably rounder by adding curvature λ in every direction, which matters most where there was almost none.

```
In [5]: a <- c(9, 0.01)           # eigenvalues of A0 = X'X / n (given)
lam <- 0.09
a_lam <- a + lam               # eigenvalues of A_lambda
c(cond_A0 = max(a) / min(a), cond_Alam = max(a_lam) / min(a_lam))
```

cond_A0: 900 cond_Alam: 90.9

(b) Show the error component in an eigendirection with eigenvalue a is multiplied by $1 - \eta(a + \lambda)$ each step. $M = 9.09$. Find the η interval that contracts every direction. Then: $\eta = 1/M \rightarrow$ two contraction factors, which direction is faster? $\eta = 2.1/M \rightarrow$ converges or not?

From Question 3(a), $\nabla L_\lambda(\boldsymbol{\beta}) = \frac{1}{n} \mathbf{X}^T \mathbf{X} \boldsymbol{\beta} - \frac{1}{n} \mathbf{X}^T \tilde{\mathbf{y}} + \lambda \boldsymbol{\beta} = \mathbf{A}_\lambda \boldsymbol{\beta} - \frac{1}{n} \mathbf{X}^T \tilde{\mathbf{y}}$, and the minimizer satisfies $\mathbf{A}_\lambda \hat{\boldsymbol{\beta}}_\lambda = \frac{1}{n} \mathbf{X}^T \tilde{\mathbf{y}}$. Subtracting the two,

$$\nabla L_\lambda(\boldsymbol{\beta}^{(k)}) = \mathbf{A}_\lambda (\boldsymbol{\beta}^{(k)} - \hat{\boldsymbol{\beta}}_\lambda) = \mathbf{A}_\lambda \mathbf{e}^{(k)}.$$

So one gradient step acts on the error as

$$\mathbf{e}^{(k+1)} = \boldsymbol{\beta}^{(k+1)} - \hat{\boldsymbol{\beta}}_\lambda = \mathbf{e}^{(k)} - \eta \mathbf{A}_\lambda \mathbf{e}^{(k)} = (\mathbf{I}_p - \eta \mathbf{A}_\lambda) \mathbf{e}^{(k)}.$$

Let $\mathbf{q}$ be an eigenvector of $\mathbf{A}_0$ with eigenvalue a . Then $\mathbf{A}_\lambda \mathbf{q} = (a + \lambda) \mathbf{q}$, so $(\mathbf{I}_p - \eta \mathbf{A}_\lambda) \mathbf{q} = (1 - \eta(a + \lambda)) \mathbf{q}$: the component of $\mathbf{e}^{(k)}$ along $\mathbf{q}$ is multiplied by

$1 - \eta(a + \lambda)$ at every step, and after k steps by $(1 - \eta(a + \lambda))^k$.

Step sizes that contract every direction. We need $|1 - \eta(a + \lambda)| < 1$ for both eigenvalues $a + \lambda \in \{9.09, 0.10\}$, i.e. $0 < \eta(a + \lambda) < 2$. The binding constraint is the largest eigenvalue $M = 9.09$, giving

$$0 < \eta < \frac{2}{M} = \frac{2}{9.09} \approx 0.220.$$

$\eta = 1/M$. Factors: $1 - \frac{9.09}{9.09} = 0$ along the large-eigenvalue direction and $1 - \frac{0.10}{9.09} \approx 0.989$ along the small one. The stiff direction is solved exactly in a single step; the flat direction shrinks by only about 1.1% per step and needs roughly 63 steps to halve its error. Convergence is governed entirely by the small eigenvalue, which is why the condition number matters.

$\eta = 2.1/M$. Factors: $1 - 2.1 = -1.1$ along the large-eigenvalue direction and $1 - 2.1 \cdot \frac{0.10}{9.09} \approx 0.977$ along the small one. The first has absolute value greater than one, so that component flips sign and grows by 10% every step. Since the initial error has a nonzero component in that direction, $\|\mathbf{e}^{(k)}\| \rightarrow \infty$: the method diverges, even though the other direction is still contracting.

```
In [6]: M <- max(a_lam) # 9.09
c(eta_max = 2 / M)
rbind(eta_1_over_M = 1 - (1 / M) * a_lam,
      eta_2.1_over_M = 1 - (2.1 / M) * a_lam) # columns: large eigenvalue, small eigenvalue
```

eta_max: 0.22002200220022

A matrix: 2 × 2 of type dbl

eta_1_over_M 0.0 0.9889989

eta_2.1_over_M -1.1 0.9768977

(c) Why is choosing λ to make gradient descent fastest not a valid statistical tuning rule?

Increasing λ improves the condition number and therefore the convergence rate of gradient descent, and in the limit $\lambda \rightarrow \infty$ every direction contracts equally fast. But the speed of the optimizer says nothing about the quality of what it converges to. The point it converges to, $\hat{\beta}_\lambda$, becomes more biased as λ grows (Question 3: each fitted direction is shrunk by $\rho_j(\lambda)$, which decreases in λ), and the fastest-converging problem, $\lambda = \infty$, converges in one step to $\hat{\beta} = \mathbf{0}$, a useless estimator.

The two criteria are also about different objects. Convergence speed depends only on the eigenvalues of $\mathbf{A}_\lambda$, i.e. on $\mathbf{X}$ and λ ; it does not involve $\tilde{\mathbf{y}}$, the noise level, or the true coefficients at all. Prediction risk is a bias-variance trade-off that depends on all of those. A rule that ignores the response cannot tell you which λ predicts well. That is why λ is chosen by an out-of-sample criterion such as cross-validation or GCV (Question 5), and the optimizer is then run to convergence at that λ , however many iterations it takes. Conditioning is an implementation concern to be handled with the step size, preconditioning, or a direct solve, not by moving the target.

Question 5: Selecting the ridge penalty for real-estate prediction

```
data/realstate.csv (414 rows, response price, 6 predictors, drop
row_id). data/realstate-split-folds.csv : split
train(332)/test(82) + cv_fold 1..10. Join on row_id. Grid  $\Lambda = \{0\} \cup$ 
```

$\{10^{(-4 + 0.05 \cdot l)} : l = 0:120\}$. Standardize with fitting-set stats only (divide by n). Center y , intercept unpenalized.

(a) 10-fold CV over Λ using the given folds. Means/scales from the 9 training folds. Mean validation MSE and $SE(\lambda) = \text{sd}(10 \text{ fold errors})/\sqrt{10}$. Plot mean MSE. $\lambda_{\min}$, then $\lambda_{1se} = \text{largest } \lambda \text{ (OLS included) with mean MSE} \leq \text{mean MSE}(\lambda_{\min}) + SE(\lambda_{\min})$.

```
In [7]: re <- read.csv("data/realstate.csv")
folds <- read.csv("data/realstate-split-folds.csv")
df <- merge(re, folds, by = "row_id")
preds <- c("date", "age", "distance", "stores", "latitude", "longitude")

train <- df[df$split == "train", ]
test <- df[df$split == "test", ]
Xtr_raw <- as.matrix(train[, preds]); ytr <- train$price
Xte_raw <- as.matrix(test[, preds]); yte <- test$price # yte is not touched until part (c)
fold_id <- train$cv_fold
c(n_train = nrow(train), n_test = nrow(test)); table(fold_id)

lambdas <- c(0, 10^(-4 + 0.05 * (0:120)))
length(lambdas)

# standardize with statistics from the fitting set only; scale divides by n
std_fit <- function(X) { mu <- colMeans(X); Xc <- sweep(X, 2, mu); list(mu = mu, s = sqrt(colMeans(
std_apply <- function(X, st) sweep(sweep(X, 2, st$mu), 2, st$s, "/")
# same ridge_fit as Question 2, but n taken from the matrix passed in
ridge_fit <- function(X, ytil, lam) {
  n <- nrow(X); p <- ncol(X)
  drop(solve(crossprod(X) + n * lam * diag(p), crossprod(X, ytil)))
}

K <- 10
cv_err <- matrix(NA, K, length(lambdas)) # fold x lambda
for (k in 1:K) {
  tr <- fold_id != k
  va <- fold_id == k
  st <- std_fit(Xtr_raw[tr, ]) # means/scales from the 9 training folds
  Xa <- std_apply(Xtr_raw[tr, ], st)
  Xv <- std_apply(Xtr_raw[va, ], st)
  b0 <- mean(ytr[tr]); ytil <- ytr[tr] - b0
  for (j in seq_along(lambdas)) {
    bhat <- ridge_fit(Xa, ytil, lambdas[j])
    cv_err[k, j] <- mean((ytr[va] - (b0 + drop(Xv %*% bhat)))^2)
  }
}
cv_mean <- colMeans(cv_err)
cv_se <- apply(cv_err, 2, sd) / sqrt(K)

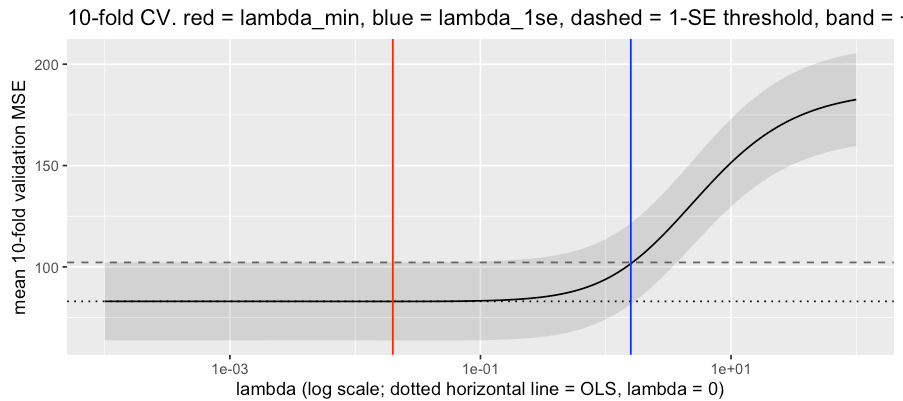
i_min <- which.min(cv_mean)
lambda_min <- lambdas[i_min]
thresh <- cv_mean[i_min] + cv_se[i_min]
lambda_1se <- max(lambdas[cv_mean <= thresh]) # largest candidate under the 1-SE line (OLS incl
c(lambda_min = lambda_min, cv_min = cv_mean[i_min], se_min = cv_se[i_min],
  lambda_1se = lambda_1se, cv_1se = cv_mean[lambdas == lambda_1se])

cv_df <- data.frame(lambda = lambdas, mse = cv_mean, se = cv_se)
ggplot(cv_df[cv_df$lambda > 0, ], aes(lambda, mse)) +
  geom_ribbon(aes(ymin = mse - se, ymax = mse + se), alpha = 0.15) +
  geom_line() +
  geom_hline(yintercept = cv_df$mse[1], linetype = "dotted") +
  geom_hline(yintercept = thresh, linetype = "dashed", colour = "grey40") +
  geom_vline(xintercept = lambda_min, colour = "red") +
  geom_vline(xintercept = lambda_1se, colour = "blue") +
  scale_x_log10() +
  labs(x = "lambda (log scale; dotted horizontal line = OLS, lambda = 0)",
    y = "mean 10-fold validation MSE",
    title = "10-fold CV. red = lambda_min, blue = lambda_1se, dashed = 1-SE threshold, band = +/-
```

n_train: 332 n_test: 82

```
fold_id
 1  2  3  4  5  6  7  8  9 10
34 34 33 33 33 33 33 33 33 33
```

lambda_min: 0.0199526231496888 **cv_min:** 82.9776493888618 **se_min:** 19.229913730506 **lambda_1se:** 1.58489319246111 **cv_1se:** 101.595531746932



On the plot the OLS candidate ($\lambda = 0$) cannot sit on a log axis, so it is drawn as the dotted horizontal line; it lies essentially on top of the curve for very small λ . $\lambda_{\min} \approx 0.020$ and $\lambda_{1se} \approx 1.58$ (the largest grid value whose mean validation MSE is under the dashed line).

(b) GCV on all 332 training rows: $GCV(\lambda) = MSE_{\text{train}} / (1 - df_{\text{eff}}/n)^2$. df_{eff} includes the intercept. Standardize once. Plot GCV, pick λ_{GCV} .

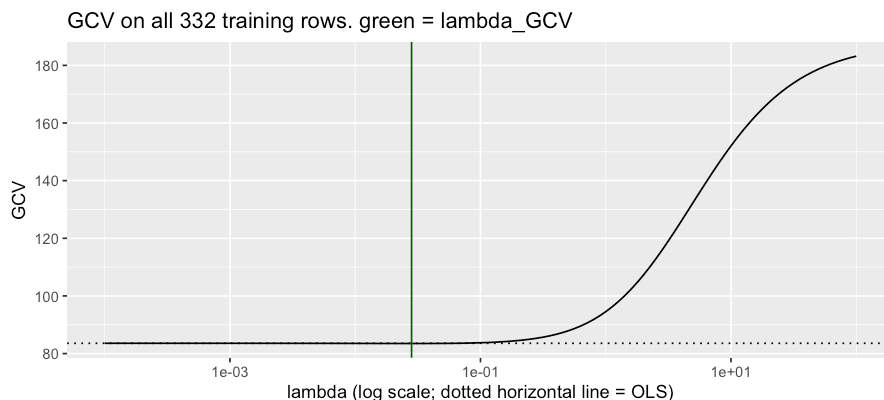
```
In [8]: st_all <- std_fit(Xtr_raw) # means/scales from all 332 training rows, kept fixed
Xtr <- std_apply(Xtr_raw, st_all)
Xte <- std_apply(Xte_raw, st_all) # test rows use the training statistics
n <- nrow(Xtr)
b0 <- mean(ytr); ytil <- ytr - b0

d2 <- svd(Xtr)$d^2
df_eff <- sapply(lambdas, function(l) 1 + sum(d2 / (d2 + n * l))) # intercept + sum of shrink
mse_train <- sapply(lambdas, function(l) mean((ytr - (b0 + drop(Xtr %*% ridge_fit(Xtr, ytil, l))))^2)
gcv <- mse_train / (1 - df_eff / n)^2

i_gcv <- which.min(gcv)
lambda_gcv <- lambdas[i_gcv]
c(lambda_gcv = lambda_gcv, gcv_min = gcv[i_gcv], df_eff = df_eff[i_gcv])

gcv_df <- data.frame(lambda = lambdas, gcv = gcv)
ggplot(gcv_df[gcv_df$lambda > 0, ], aes(lambda, gcv)) +
  geom_line() +
  geom_hline(yintercept = gcv[1], linetype = "dotted") +
  geom_vline(xintercept = lambda_gcv, colour = "darkgreen") +
  scale_x_log10() +
  labs(x = "lambda (log scale; dotted horizontal line = OLS)", y = "GCV",
       title = "GCV on all 332 training rows. green = lambda_GCV")
```

lambda_gcv: 0.0281838293126445 **gcv_min:** 83.5019140202536 **df_eff:** 6.68865672231468



(c) Refit OLS, $\lambda_{\min}$, λ_{1se} , λ_{GCV} on all training rows. Table: train MSE, test MSE, df_{eff} . Interpret. Why must you not pick the row with the smallest test MSE?

```
In [9]: # all three lambdas are fixed above, before the test responses are used
choices <- c(OLS = 0, lambda_min = lambda_min, lambda_1se = lambda_1se, lambda_GCV = lambda_gcv)

final <- do.call(rbind, lapply(names(choices), function(nm) {
  l <- choices[[nm]]
  bhat <- ridge_fit(Xtr, ytil, l)
  data.frame(model = nm,
    lambda = l,
    train_mse = mean((ytr - (b0 + drop(Xtr %*% bhat)))^2),
    test_mse = mean((yte - (b0 + drop(Xte %*% bhat)))^2),
    df_eff = 1 + sum(d2 / (d2 + n * l)))
}))
knitr::kable(final, digits = 3)
```

model	lambda	train_mse	test_mse	df_eff
:-----:	-----:	-----:	-----:	-----:
OLS	0.000	80.094	66.322	7.000
lambda_min	0.020	80.135	66.000	6.773
lambda_1se	1.585	100.374	81.200	3.033
lambda_GCV	0.028	80.171	65.903	6.689

Interpretation. $\lambda_{\min}$ and λ_{GCV} land almost on top of each other (about 0.020 and 0.028) and both are tiny: their effective degrees of freedom are about 6.8 and 6.7 against 7 for OLS, and their training and test MSEs are within half a unit of OLS. With 332 observations and only 6 predictors that are not strongly collinear, there is little variance for ridge to remove, so CV and GCV both say "barely shrink". The 1-SE choice is very different: $\lambda_{1\text{se}} \approx 1.58$ cuts the effective degrees of freedom to about 3, and training MSE rises by about 25% and test MSE by about 22%. The CV curve is flat near its minimum and its standard error is large (about 19 on a minimum of about 83, ten folds of 33 noisy house prices each), so the 1-SE band reaches far to the right and admits a penalty that is clearly too heavy here. The 1-SE rule buys simplicity at a real cost in this data set.

Why not pick the row with the smallest test MSE. The test set exists to give an honest estimate of the error of a model that was chosen *without* it. The moment the test responses are used to choose between the four rows, they become part of the training procedure, and the winning row's test MSE is an optimistically biased estimate of its true error, because with only 82 test points the differences among OLS, $\lambda_{\min}$ and λ_{GCV} (66.3, 66.0, 65.9) are far inside the noise of a single test split. Picking the minimum would be selecting on that noise, and we would then have no data left on which to measure how well the selected model actually predicts. That is why the three penalties were fixed by CV and GCV on the training data first, and the test responses were examined only once, afterwards, to report the errors.

explain-to-me skill

Saved as `explain-to-me/SKILL.md` and installed at `.claude/skills/explain-to-me/SKILL.md` in the course repository (Claude Code's project-level skill location). Tested by asking the agent to use `explain-to-me` on Question 2.

name: explain-to-me

description: Explain what a STAT 432 homework question is asking, without solving it. Use only when Owen explicitly says "use explain-to-me" on a question.

explain-to-me

Intent: I want to understand the question before I touch it. Help me

read it, not do it.

When I ask you to use this skill on a homework question:

1. Restate the goal of the question in two or three plain sentences. Say what the question is really testing.
2. List the concrete deliverables, part by part: what has to be computed, plotted, derived, or argued.
3. Name the inputs and constraints I must respect (seed, data files, standardization rule, penalty grid, what must stay fixed, what must not be touched).
4. Point me to the Week 3 lecture section that covers each part:
 - Ridge Regression: Stability Through Shrinkage –
<https://teazrq.github.io/stat432rpy/topics/ridge-regression/ridge-regression.html>
 - From a Penalized Objective to a Fitted Ridge Model –
<https://teazrq.github.io/stat432rpy/topics/ridge-regression/optimization-and-cross-validation.html>
5. Flag one or two places where students usually go wrong on this kind of question.
6. End with a question back to me that checks I understood, then stop.

Rules:

- Do not write the solution. No code, no derivations, no final numbers, no completed interpretation.
- Do not open the solutions file even if it is in the folder.
- Keep it under about 250 words per question. Plain words over jargon.
- If the question is ambiguous, say what the two readings are instead of picking one for me.
- Only run when I explicitly invoke the skill. Do not trigger it on every homework mention.